

# Parallel Programming

## Assignment 4: Parallel Models

### Spring Semester 2026

Assigned on: **11.03.2026**

**Due by: (Wednesday Exercise) 16.03.2026**  
**(Friday Exercise) 18.03.2026**

## Task 1 – Pipelining

Bob, Mary, John and Alice share a flat. In this flat they share a washing machine, a dryer and an ironing board. The washing machine takes 50 minutes for one wash cycle. The dryer takes 90 minutes. Everyone of them takes roughly 15 minutes to iron their laundry.

- a) Assuming they would do their laundry in strictly sequential order (one person starts only after the other finished ironing), calculate how long would it take to finish the laundry.
  
  
  
  
  
  
  
  
  
  
- b) Are there any better options? If yes, describe them and calculate the improved laundry time, as well as the speedup of this version compared to the strictly sequential version. Further, determine whether this pipeline is balanced or unbalanced.
  
  
  
  
  
  
  
  
  
  
- c) Can you devise a better strategy assuming that the four roommates bought another dryer? If yes, calculate the new laundry time. Further, determine whether such an improved pipeline is balanced or unbalanced and calculate the pipeline throughput as well as latency.

## Task 2 – Pipelining II

Instruction pipelining is a form of parallelism (also called instruction level parallelism) commonly used to improve program performance. The main idea is that the execution of multiple instructions can be partially overlapped which leads to reduction of the overall time required to complete the execution.

- a) Let us consider a following for loop:

```
for (int i = 0; i < data.length; i++) {  
    data[i] = data[i] * data[i];  
}
```

Assume that the multiplication has a throughput of 1 instruction per cycle and a latency of 6 cycles. That is, the processor can in each cycle start executing a single new multiplication instruction and it will take the processor 6 cycles to compute the result. Further, consider only arithmetic operations performed in the loop body (i.e., in this case `data[i] * data[i]`) and ignore all the other operations (e.g, storing the result, incrementing the loop counter, evaluating the loop termination condition). However, all the computation in the loop body must be finished in order to start a new loop iteration.

Calculate how many cycles the processor needs to execute the loop a) given the above simplifying assumptions.

- b) Let us consider a different loop that calculates the same result as the loop in a), assuming that the array length is divisible by two:

```
for (int i = 0; i < data.length; i += 2) {  
    j = i + 1;  
    data[i] = data[i] * data[i];  
    data[j] = data[j] * data[j];  
}
```

Assume that the addition has a throughput of 1 instruction per cycle and a latency of 3 cycles. Note that it is allowed for the processor to issue a multiplication instruction even if the addition instruction is still being computed and vice versa (i.e., we can issue an addition in the first cycle followed by multiplication in cycle 2). Calculate how many cycles the processor needs to execute the loop b).

- c) Finally, let us consider the following loop that calculates the same results as the loops in a) and b), assuming that the array length is divisible by four:

```
for (int i = 0; i < data.length; i += 4) {  
    j = i + 1;  
    k = i + 2;  
    l = i + 3;  
    data[i] = data[i] * data[i];  
    data[j] = data[j] * data[j];  
    data[k] = data[k] * data[k];  
    data[l] = data[l] * data[l];  
}
```

Calculate how many cycles the processor needs to execute the loop c).

**Note:** The above optimization is called loop unrolling and is typically performed automatically by the compiler. You should not write such code manually unless you are writing performance critical code and are sure that the compiler cannot perform this optimization automatically.

### Task 3 – Identify Potential Parallelism

- a) Inspect the code snippets of the two `for` loops below. For each loop explain if it is OK to use a parallel `for` loop instead.

**Note:** A parallel `for` loop is a parallel programming construct covered in Exercise 2 that allows different loop iterations to be performed in parallel. This should not be confused with instruction level parallelism.

(a) Loop-1

```
for (int i=1; i<size; i++) {    // for loop: i from 1 to (size-1)
    if (data[i-1] > 0)          // If the previous value is positive
        data[i] = (-1)*data[i]; // change the sign of this value
    }                          // end for loop
```

(b) Loop-2

```
for (int i=0; i<size; i++) {    // for loop: i from 0 to (size-1)
    data[i] = Math.sin(data[i]); // calculate sin() of the value
    }                          // end for loop
```

## **Submission**

There is no submission for this exercise.